Conception technique et adaptation au monde réel

1. Choix de l'hébergement (comparaison réalisée le 27/10/2026)

Critère	**Cloudflare Workers + Durable Objects (retenu)**	**Supabase (Postgres + Realtime)**	**Serveur Node maison / VPS**
Gratuité durable	Offre « Workers Free » sans durée limitée	Offre gratuite, mais **projet mis en pause après 7 jours d'inactivité** (à réactiver à la main)	VPS gratuits rares, souvent avec carte bancaire
Temps réel	WebSocket natif, un objet par partie	Realtime (200 connexions simultanées sur l'offre gratuite selon des sources secondaires)	à coder et héberger
Logique serveur de confiance	Le Durable Object **est** le serveur : il applique toutes les règles	Il faudrait des fonctions Postgres/Edge + des politiques RLS fines pour cacher les rôles	oui
Cohérence	Un seul fil d'exécution par partie : pas de course entre deux votes simultanés	transactions SQL à écrire avec soin	à gérer
Maintenance	aucune machine, aucun système à mettre à jour	faible	élevée
Sous-domaine gratuit HTTPS	`*.workers.dev`	`*.supabase.co` (+ hébergement du front ailleurs)	non

Quotas gratuits vérifiés dans la documentation officielle Cloudflare :

- Workers : 100 000 requêtes/jour (remise à zéro à 0 h UTC). Les **fichiers statiques sont gratuits et illimités** et n'invoquent pas le Worker.
- Durable Objects (backend SQLite, seul disponible en gratuit depuis le 07/04/2025) : 100 000 requêtes/jour, 13 000 Go·s de durée/jour, 5 millions de lignes lues et 100 000 lignes écrites par jour, 5 Go de stockage. Les messages WebSocket **entrants** comptent pour 1/20 de requête ; les messages sortants ne sont pas facturés ; un objet inactif (hibernation) n'est pas facturé en durée.
- Dépassement : les opérations échouent (pas de facturation surprise sur l'offre gratuite).

Dimensionnement pour une soirée de 40 joueurs, 2 heures :

- Connexions : ~40 WebSockets + le MJ ; reconnexions incluses, quelques centaines de requêtes.
- Messages entrants : ~40 joueurs × ~4 actions/min × 120 min ≈ 19 000 messages → **~1 000 requêtes** équivalentes. Les « ping » de maintien de connexion reçoivent une réponse automatique sans réveiller l'objet.
- Écritures : une ligne par action enregistrée ≈ 20 000 lignes au pire → sous les 100 000/jour.
- Durée : même sans hibernation, 128 Mo × 7 200 s ≈ 920 Go·s → **7 % du quota quotidien**.
- Taille de l'état : < 300 Ko (limite par valeur : 2 Mo). → Une partie normale utilise quelques pourcents des quotas. Plusieurs parties le même jour restent possibles. **Ce qui imposerait de**

changer de solution : des dizaines de parties de 40 joueurs le même jour, ou un usage public massif de la création de parties.

Compte requis : la mise en ligne exige un compte Cloudflare gratuit (création par e-mail). Nous n'avons pas trouvé dans la documentation officielle de mention explicite « sans carte bancaire » ; à notre connaissance l'offre Workers Free n'en demande pas. Aucun service payant, aucun domaine, aucune clé d'API externe n'est utilisé.

2. Architecture

```
Téléphones (PWA Preact)  ⇄  WebSocket  ⇄  Worker Cloudflare  →  Durable Object « GameRoom »
(1 par partie)
        │                                       │                       │  ├─ moteur de règles
(TypeScript partagé)
        └── fichiers statiques (gratuits) ──────┘                       │  ├─ état persistant
(SQLite)
                                                                        │  └─ alarme
(minuteurs de réunion, sabotages…)
```

- `src/shared/` : **moteur de règles pur** (aucune dépendance réseau), utilisé par le serveur et testé unitairement ; génération des vues filtrées ; catalogue des tâches ; génération des énigmes imprimées.
- `src/worker/` : point d'entrée HTTP (`/api/games` pour créer, `/api/games/CODE/ws` pour le temps réel) et Durable Object.
- `src/client/` : application mobile (Preact + Vite, ~65 Ko compressés + lecteur QR chargé à la demande).

Le navigateur n'est jamais cru. Chaque message est une *intention* ; le serveur vérifie l'identité (jeton aléatoire dont seul le hachage SHA-256 est stocké), l'appartenance à la partie, le rôle, le statut vivant/mort, la phase, les délais, la validité des codes, l'unicité des votes, puis renvoie à chaque appareil **uniquement** ce qu'il a le droit de voir.

États de partie

```
lobby ──start──▶ playing ◀──────────────┐
                   │  signalement / bouton d'urgence / MJ
                   ▼                    │
                meeting : gather → discussion → voting → result ──┘
                   │                                  (vérification des victoires)
playing/meeting ──victoire / fin MJ──▶ ended ──newGame──▶ lobby
pause / arrêt d'urgence : gèle tous les minuteurs (décalés à la reprise), refuse toute action
des joueurs.
```

Synchronisation et reprise

- Après chaque action acceptée : état sauvegardé, puis diffusion d'une vue filtrée à chaque téléphone. Les comptes à rebours sont des **instants absolus** du serveur ; chaque téléphone corrige le décalage de son horloge.
- Les minuteurs (fin de rassemblement/discussion/vote, rassemblement raccourci à 15 s dès que la majorité est arrivée, réacteur, oxygène, disparition des corps) sont gérés par l'**alarme**

du Durable Object : ils avancent même si personne n'agit.
- Une action n'est affichée comme réussie qu'après l'accusé de réception du serveur. Hors connexion, l'application affiche un bandeau rouge et refuse d'envoyer ; aucune file d'attente hors ligne trompeuse.
- Rechargement / perte de réseau : reconnexion automatique (délais croissants), réauthentification par jeton, renvoi de l'état complet. L'état d'un mort reste mort.
- Changement de téléphone : le MJ génère un code de reconnexion (15 min, usage unique) ; l'ancien appareil est déconnecté.
- Perte du MJ : la partie continue (les minuteurs tournent sur le serveur) ; le MJ se reconnecte avec son jeton, ou depuis un autre appareil avec le **code maître du jeu** (8 caractères, 8 essais par 10 min).

Alertes et notifications

- Application ouverte : écran d'alerte plein écran, sons distincts (Web Audio), vibration (API Vibration, Android).
- Application en arrière-plan : le client signale sa visibilité au serveur (`vis`) ; à chaque événement public (début, réunion, sabotage critique, STOP, fin, test d'alerte ; alertes sécurité pour le MJ), le Durable Object envoie une notification **Web Push** aux seuls appareils abonnés qui ne sont pas au premier plan.
- Clés VAPID générées automatiquement à la création de chaque partie et stockées avec elle (aucun secret à configurer) ; contenu chiffré aes128gcm (RFC 8291), testé avec le vecteur officiel ; seuls les services de push des navigateurs (Google, Apple, Mozilla, Microsoft) sont autorisés comme destinataires, pour ne pas transformer le serveur en relais.
- Limite de l'offre gratuite : 50 sous-requêtes par invocation → au plus 45 notifications par événement. Les abonnements expirés (réponse 404/410) sont supprimés.

3. Sécurité

- Codes de partie : 6 caractères sur un alphabet sans ambiguïté ($27^6 \approx 387$ millions). Codes de postes : 4 caractères ($31^4 \approx 923\,000$). Codes d'élimination et de corps : 4 chiffres, uniques parmi les codes actifs, durée de vie courte.
- Anti-force brute : 6 codes erronés en 60 s → blocage de 60 s, consigné dans le journal secret du MJ.
- Limitation de débit : 60 messages / 10 s par connexion.
- Les rôles, codes de postes, réponses des défis, jetons et le journal secret ne quittent jamais le serveur, sauf vers le MJ authentifié. Tests automatisés de non-fuite sur la vue de chaque joueur.
- Aucune clé secrète n'est nécessaire au fonctionnement : il n'y en a donc aucune dans le dépôt. Le jeton Cloudflare de déploiement est un **secret GitHub** (jamais dans le code).
- Entrées utilisateur nettoyées et bornées (pseudonymes 2–16 caractères sans balises, terrain et paramètres validés côté serveur).
- Données personnelles : aucune. Pas de compte, pas de géolocalisation, pas de micro ; la caméra n'est ouverte qu'à la demande pour lire un QR code, et rien n'est enregistré.

4. Mécanique d'élimination : analyse des options

Méthode	Immersion	Simplicité	Discrétion	Anti-triche	Verdict
A. La victime appuie seule sur « Je suis mort »	bonne	excellente	bonne	**nulle** : le délai d'élimination ne peut pas être vérifié, fausses déclarations possibles	rejetée
B. La victime choisit son tueur dans une liste	bonne	bonne	bonne	**fuite** : un refus révélerait qu'un joueur n'est pas imposteur	rejetée
C. L'imposteur scanne un QR code affiché par la victime	moyenne	faible (la victime doit sortir son téléphone, ouvrir l'écran)	faible	bonne	rejetée
D. Bluetooth / NFC entre téléphones	—	—	—	—	**impossible** sur iPhone depuis une page web
E. Code d'élimination à usage unique (version 1) : l'imposteur arme un code à 4 chiffres, touche l'épaule de la victime et lui montre l'écran ; la victime le saisit	bonne	**faible en pratique** : 15 à 40 s (déverrouiller, reconnexion, onglet Urgence, abandonner un éventuel mini-jeu, saisir avant 45 s)	**faible** : deux personnes penchées ensemble sur leurs téléphones, code rouge visible des témoins	bonne	remplacée
F. L'imposteur choisit le pseudonyme de sa victime (retenu) : il la touche, murmure « éliminé·e », touche son pseudonyme puis « Valider » ; le téléphone de la victime passe seul en écran « corps » (vibration, sans son)	**bonne** (geste court, la victime s'assoit aussitôt)	**excellente** : 2 touchers pour l'imposteur, rien pour la victime	**bonne** : panneau neutre, aucun échange de téléphones	**moyenne** : délai, cible vivante et non-alliée vérifiés par le serveur ; la proximité repose sur l'honnêteté, avec **contestation** par la victime et arbitrage du MJ	**retenue**

Pourquoi F ne reproduit pas la fuite de B : c'est l'**imposteur** qui choisit, et il connaît déjà ses alliés ; le refus « c'est un allié » ne lui apprend rien. Un mort non encore annoncé est refusé sans consommer le délai ; le tueur voit sa propre victime disparaître de sa liste, ses alliés ne l'apprennent qu'à la réunion.

Limites honnêtes : le serveur ne peut pas prouver que les deux personnes étaient côte à côte. Un imposteur pourrait choisir un pseudonyme à distance : c'est interdit par le règlement, la « victime » s'en aperçoit aussitôt (son téléphone vibre sans qu'on l'ait touchée) et touche **« Je n'ai pas été touché·e (contester) »** (une seule fois par mort) ; le MJ reçoit une alerte « CONTESTATION » avec le nom de l'imposteur (absent de la notification d'écran verrouillé). Une erreur de pseudonyme se corrige de la même façon. **Arbitrage** : le MJ écoute les deux joueurs (en privé), consulte le journal (heure de l'élimination et de la contestation) et peut annuler une élimination (« Annuler la mort ») ou déclarer une mort ; chaque correction est consignée.

Consentement : le contact se limite à une tape légère sur l'épaule ou le bras ; un joueur peut demander en début de partie à être « éliminé sans contact » (l'imposteur se place à portée de bras et dit « éliminé »).

5. Ce que la technologie ne peut pas automatiser — et la procédure prévue

Point	Procédure
Proximité réelle lors d'une élimination	geste physique + code montré ; arbitrage du MJ
Présence réelle à un poste	code/QR posé à un endroit fixe ; interdiction de photographier ; journal des codes erronés
Silence des fantômes	badge « FANTÔME » visible, règle de silence, exclusion possible par le MJ
Déplacements interdits (zones, conduits réservés)	étiquettes imprimées, présentation avant la partie, règle d'honneur
Arrivée en salle de réunion	bouton « Je suis arrivé·e » + minuteur ; le MJ peut passer à l'étape suivante
Téléphone en panne / déchargé	le MJ déclare la mort, signale un corps, convoque une réunion, ou donne un code de reconnexion pour un autre téléphone
Sécurité physique	consignes affichées à tous, arrêt d'urgence du MJ, bouton « Alerte sécurité » pour chaque joueur